School of Future Tech

Case Study Report

on

Case study 145 :

SMART CRIME ANALYTICS & POLICE INVESTIGATION SYSTEM

by

Yuvraj Jitendra Singh (150096725009)

Student Details

Name:-Yuvraj Jitendra Singh
Roll Number:-150096725009
College:-ITM Skills University,Kharghar
Batch:-2025-2029
Course:BTech (CSE)

Git hub repository link :-
https://github.com/ImYuv19/SMART-CRIME-ANALYTICS-POLICE-INVESTIGATION-SYSTEM

Index

---

1. Introduction to the Case Study

Crime investigation and law enforcement agencies manage large amounts of crime-related information such as crime cases, criminal records, investigation activities, and crime reports. Efficient handling of such information is important for quick decision-making and investigation processes.

This case study focuses on developing a Smart Crime Analytics & Police Investigation System using Data Structures and Algorithms (DSA) in C++. The system demonstrates how different DSA concepts can be applied in a real-world law enforcement environment.

The project provides functionalities for crime case management, criminal record storage, crime report processing, investigation rollback procedures, criminal network analysis, and crime analytics. The implementation is completely console-based and developed using C++.

---

2. Problem Statement / Case Background (Abstract)

Law enforcement departments often need efficient systems to manage crime cases, criminal databases, and investigation activities. Traditional manual systems can be slow and inefficient when handling large volumes of records.

The objective of this project is to develop a Smart Crime Analytics & Police Investigation System that utilizes various Data Structures and Algorithms to improve data management and analysis. The system enables crime case storage, report processing, suspect identification, criminal network analysis, and crime pattern recognition.

The project demonstrates practical implementation of Sorting Algorithms, Searching Algorithms, Stack, Queue, Binary Search Tree (BST), AVL Tree, Hashing, Breadth First Search (BFS), and Depth First Search (DFS) in solving real-world crime management problems.

---

3. Problem Statement / Case Study Design

The system is designed as a menu-driven console application consisting of multiple functional modules.

Module 1: Crime Case Management

Stores and displays crime case information including:

- Case ID
- Crime Type
- Severity Level
- Investigation Status

Module 2: Sorting Module

Ranks crime cases based on severity level using Bubble Sort.

Module 3: Searching Module

Retrieves crime cases and criminal records using Linear Search.

Module 4: Queue Module

Processes incoming crime reports using FIFO (First In First Out) methodology.

Module 5: Stack Module

Maintains investigation history and supports rollback operations using LIFO (Last In First Out).

Module 6: Binary Search Tree (BST)

Stores criminal records in a hierarchical structure for efficient retrieval.

Module 7: AVL Tree

Stores criminal records using a self-balancing binary search tree to maintain optimal search performance.

Module 8: Hashing Module

Provides rapid suspect identification using hash table techniques.

Module 9: Criminal Network Analysis

Represents relationships between criminals using graph structures and performs BFS and DFS traversals.

Module 10: Crime Analytics

Generates crime statistics, crime patterns, and predictive insights.

---

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

The project utilizes the following Data Structures and Algorithms:

Bubble Sort

Used to rank crime cases according to severity level.

Time Complexity:

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Linear Search

Used for searching crime cases and criminal records.

Time Complexity:

- Best Case: O(1)
- Worst Case: O(n)

Stack

Used for investigation rollback procedures.

Characteristics:

- LIFO (Last In First Out)

Operations:

- Push
- Pop
- Peek

Queue

Used for crime report processing.

Characteristics:

- FIFO (First In First Out)

Operations:

- Enqueue
- Dequeue

Binary Search Tree (BST)

Used for criminal record storage.

Advantages:

- Organized hierarchical structure
- Efficient searching and insertion

Average Time Complexity:

- Search: O(log n)
- Insert: O(log n)

AVL Tree

Used to maintain balanced criminal databases.

Advantages:

- Self-balancing tree
- Guaranteed O(log n) operations

Operations:

- Left Rotation
- Right Rotation
- Double Rotations

Hashing

Used for rapid suspect identification.

Advantages:

- Fast lookup operations
- Average O(1) search time

Collision Handling:

- Separate Chaining

Graph

Used for criminal network representation.

Graph Traversals:

- Breadth First Search (BFS)
- Depth First Search (DFS)

Time Complexity:

- BFS: O(V + E)
- DFS: O(V + E)

Where:

- V = Number of Vertices
- E = Number of Edges

---

5. Problem Statement / Case Study Implementation Details and Snapshots

Development Environment

Language:

- C++

Platform:

- Console-Based Application

Compiler:

- GCC / G++

Functional Implementation

Crime Case Management

Users can:

- Add crime cases
- View crime cases

Crime Report Processing

Users can:

- Add reports
- Process reports
- View pending reports

Investigation Rollback

Users can:

- Add investigation actions

- Undo actions
- View investigation history

Criminal Database

Implemented using:

- BST
- AVL Tree
- Hash Table

Users can:

- Insert criminal records
- Search criminal records
- Display records

Criminal Network Analysis

Users can:

- Add criminal connections
- Perform BFS traversal
- Perform DFS traversal

Crime Analytics Dashboard

Displays:

- Total Cases
- Open Cases
- Closed Cases
- Highest Severity Crime
- Most Frequent Crime Type

Snapshots

```
1. Add Crime Case
2. View Crime Cases
3. Back to Main Menu
Enter Your Choice: 2

================================================================
                    CRIME CASES REGISTERED
================================================================
  Case ID   |   Crime Type   |   Severity Level   |   Status
----------------------------------------------------------------
  101       |   Theft        |   3                |   Closed
  102       |   Homicide     |   10               |   Open
  103       |   Cybercrime   |   6                |   Open
  104       |   Assault      |   5                |   Open
  105       |   Fraud        |   4                |   Closed
================================================================
```

```
==========================================
                MAIN MENU
==========================================
1.  Crime Case Management
2.  Sort Cases by Severity
3.  Search Records
4.  Crime Reports Queue
5.  Investigation History
6.  Criminal Database (BST)
7.  Criminal Database (AVL)
8.  Suspect Lookup (Hash)
9.  Criminal Network (Graph)
10. Crime Analytics Dashboard
11. Exit System
==========================================
```

---

6. Problem Statement / Case Study Results and Conclusion

Results

The Smart Crime Analytics & Police Investigation System was successfully developed and tested.

The application successfully demonstrates:

- Crime Case Management
- Crime Report Processing
- Investigation Rollback
- Criminal Record Storage

- Rapid Suspect Identification
- Criminal Network Analysis
- Crime Analytics and Crime Pattern Recognition

All required Data Structures and Algorithms were implemented and verified successfully.

Conclusion

The project demonstrates the practical application of Data Structures and Algorithms in solving real-world crime management problems.

Through the implementation of Bubble Sort, Linear Search, Stack, Queue, BST, AVL Tree, Hashing, BFS, and DFS, the system provides efficient management of crime-related data and analysis.

The project successfully fulfills all case study requirements and highlights the importance of DSA concepts in designing efficient software solutions for law enforcement and public safety systems.

---

7. References

1. Data Structures and Algorithms using C++ – Reema Thareja.
2. Introduction to Algorithms – Thomas H. Cormen.
3. Data Structures Using C++ – D.S. Malik.
4. GeeksforGeeks DSA Learning Resources.
5. C++ Documentation and Standard Library Reference.
6. Course Notes and Case Study Guidelines provided by ITM Skills University.